

CS 4472 / LABORATORY REPORT 02

RSA Public-Key Encryption and Signatures

David Nunes-Childs

OVERVIEW

This lab works through every step of the RSA algorithm on real numbers using the OpenSSL `BIGNUM` library. I derived a private key from its prime factors, encrypted and decrypted a message, produced a signature by direct exponentiation, and verified a signature both intact and after a one-bit corruption. Each task is a standalone C program compiled against `libcrypto`; the arithmetic is 256-bit throughout, which is far below a secure modulus size but keeps every intermediate value readable.

M Method and source files

Building and running each task

Every task is a single translation unit linked against OpenSSL's crypto library. No build system is involved:

```
gcc bn_lab_task1.c -lcrypto -o task1 && ./task1
```

The handout writes this as `sudo gcc`. Elevation is not needed — compilation reads the system headers and writes only into the working directory. The screenshots show `sudo` because I followed the handout verbatim before noticing.

Source files

File	Contents
<code>bn_lab_task1.c</code>	Task 1 — derives <code>d</code> from <code>p</code> , <code>q</code> , and <code>e</code>
<code>bn_lab_task2.c</code>	Task 2 — encrypts <code>A top secret!</code> and decrypts the result as a check
<code>bn_lab_task3.c</code>	Task 3 — decrypts the supplied ciphertext
<code>bn_lab_task4.c</code>	Task 4 — signs a message directly and verifies the signature
<code>bn_lab_task5.c</code>	Task 5 — verifies Alice's signature, intact and corrupted
<code>example/bn_sample.c</code>	The handout's worked <code>BIGNUM</code> example, kept for reference
<code>bn_lab.c</code>	Scratch file used while learning the API; superseded by <code>bn_lab_task1.c</code>

01 Deriving the private key

Compute d from p, q, and e

The public exponent e and the two primes are given. The private exponent is the modular inverse of e with respect to Euler's totient, so the program computes $n = p \cdot q$, then $\phi(n) = (p-1)(q-1)$, confirms that e and $\phi(n)$ are coprime, and inverts:

```
BN_mul(n, p, q, ctx);           // n = p * q   (before p and q are modified)
BN_sub_word(p, 1);              // p = p - 1
BN_sub_word(q, 1);              // q = q - 1
BN_mul(totient_n, p, q, ctx);    // phi(n) = (p-1)(q-1)

BN_gcd(g, e, totient_n, ctx);
assert(BN_is_one(g));            // e must be coprime to phi(n)

BN_mod_inverse(d, e, totient_n, ctx); // d = e^-1 mod phi(n)
```

Implementation note: `BN_sub_word()` decrements its argument in place rather than returning a new value. n therefore has to be computed *before* the two decrements. In an earlier draft I reordered these lines and silently ended up inverting e modulo $\phi(n)$ while labelling the result $p \cdot q$. The printed value happened to be correct, but the variable name no longer described it.

Results

Quantity	Value (hexadecimal)
p	F7E75FDC469067FFDC4E847C51F452DF
q	E85CED54AF57E53E092113E62F436F4F
e	0D88C3 (decimal 886,979)
n = p · q	E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1
d	3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB

The program prints the key pair in decimal. Two 128-bit primes give a 256-bit modulus, so n and d are 78-digit decimal numbers:

```
pub: (886979,
      101776877529005912638346811918779931246783058062684819617574643018368103302097)
priv: (24212225287904763939160097464943268930139828978795606022583874367720623008491,
       101776877529005912638346811918779931246783058062684819617574643018368103302097)
```

```
* lab2 sudo gcc bn_lab_task1.c -lcrypto -o task1 && ./task1
pub: (886979, 101776877529005912638346811918779931246783058062684819617574643018368103302097)
priv: (24212225287904763939160097464943268930139828978795606022583874367720623008491, 101776877529005912638346811918779931246783058062684819617574643018368103302097)
```

Figure 1. Compiling and running `bn_lab_task1.c`. The modulus is shared by both keys; only the exponent differs.

02 Encrypting a message

Tasks 2 through 4 use a different key pair, supplied by the handout, with $e = 010001$ (decimal 65,537). The private exponent d is also given so the result can be checked.

Convert the plaintext to an integer

RSA operates on integers, so the ASCII string has to become one. The handout's Python 2 command `print("A top secret!".encode("hex"))` no longer works under Python 3, where `str.encode()` returns `bytes` and the hex codec was removed. The Python 3 equivalent is:

```
python3 -c 'print("A top secret!".encode("utf-8").hex())'
4120746f702073656372657421
```

Those 13 bytes are loaded with `BN_hex2bn()` and treated as a 104-bit integer. This is comfortably smaller than the 256-bit modulus, which matters: RSA is only invertible when $M < n$, since the ciphertext is a residue modulo n and anything larger would be reduced and lost.

Encrypt and verify

Encryption is a single modular exponentiation, $C = M^e \bmod n$. I decrypted the result in the same program to confirm the round trip:

```
BN_mod_exp(C, M, e, n, ctx);    // C = M^e mod n
BN_mod_exp(res, C, d, n, ctx);  // res = C^d mod n, expected to equal M
```

```
ciphertext (hex): 6FB078DA550B2650832661E14F4F8D2CFAEF475A0DF3A75CACDC5DE5CFC5FADC
plaintext (hex): 4120746F702073656372657421
```

Converting the recovered value back to ASCII returns `A top secret!`, so the derived ciphertext is correct.

Observation: this is textbook RSA, with no padding. The transformation is deterministic, so the same plaintext under the same public key always produces the identical ciphertext. Against a small or guessable message space an attacker can simply encrypt candidate plaintexts and compare. Deployed systems randomise the encoding first, as specified in [RFC 8017 § 7.1 \(RSAES-OAEP\)](#).

```
→ lab2 sudo gcc bn_lab_task2.c -lcrypto -o task2 && ./task2
ciphertext (hex): 6FB078DA550B2650832661E14F4F8D2CFAEF475A0DF3A75CACDC5DE5CFC5FADC
plaintext (hex) 4120746F702073656372657421
→ lab2 python3 -c 'print(bytes.fromhex("4120746F702073656372657421").decode("utf-8"))'
A top secret!
```

Figure 2. Encryption followed by an in-program decryption check. The final Python command converts the recovered hexadecimal back to ASCII.

03 Decrypting a message

The key pair is unchanged from Task 2. Decryption is the same primitive as encryption with the private exponent substituted for the public one, $M = C^d \bmod n$:

```
BN_hex2bn(&C, "8C0F971DF2F3672B28811407E2DABBE1DA0FEBBDFC7DCB67396567EA1E2493F");
BN_mod_exp(M_decoded, C, d, n, ctx);
```

```
plaintext (hex): 50617373776F72642069732064656573
```

Converting those 16 bytes to ASCII gives the recovered message:

```
python3 -c 'print(bytes.fromhex("50617373776F72642069732064656573").decode("ascii"))'
Password is dees
```

Why encryption and decryption are the same operation

Both directions call `BN_mod_exp()`; only the exponent changes. This works because `d` was chosen as the inverse of `e` modulo $\phi(n)$, so $e \cdot d \equiv 1 \pmod{\phi(n)}$ and therefore $(M^e)^d \equiv M \pmod{n}$ by Euler's theorem. The exponents are interchangeable, which is exactly what Task 4 exploits to build a signature.

Decryption always produces something

Raw RSA carries no padding and no integrity check, so every integer below `n` is a well-formed ciphertext and decrypts to *some* integer. `BN_mod_exp()` has no way to report a wrong key or a corrupted input; it returns a residue either way. Detecting that the result is meaningless is left entirely to whatever interprets it — here, my reading of the ASCII decoding. This is exactly what Task 5 runs into: a corrupted signature does not raise an error, it simply yields bytes that do not spell anything. Padded schemes such as those in [RFC 8017](#) add structure to the encoded message precisely so that a failure is detectable rather than merely implausible.

Observation: the plaintext is `Password is dees`, not `A top secret!`. The handout shows the string `A top secret!` alongside a hex-to-ASCII command in this section, but that snippet only demonstrates the conversion syntax carried over from Task 2 — it is not the answer to this task. Decrypting the supplied ciphertext is the only way to know which it is, and that is the point of the exercise.

```
→ lab2 sudo gcc bn_lab_task3.c -lcrypto -o task3 && ./task3
plaintext (hex): 50617373776F72642069732064656573
→ lab2 python3 -c 'print(bytes.fromhex("50617373776F72642069732064656573").decode("ascii"))'
Password is dees
```

Figure 3. Decryption of the supplied ciphertext. The recovered plaintext is `Password is dees`.

04 Signing a message

The handout asks for the message to be signed directly rather than by signing its hash, which makes the underlying operation visible: a signature is the message raised to the *private* exponent, $S = M^d \bmod n$. Anyone holding the public key can raise S to e and recover M , which is the verification step.

```
BN_mod_exp(C, M, d, n, ctx);    // sign   with the private exponent
BN_mod_exp(res, C, e, n, ctx);  // verify with the public exponent
```

Signing two nearly identical messages

I signed `I owe you $2000.` and then changed the amount to `$1500.` and signed again. The handout suggests `$3000`; any small edit demonstrates the same effect.

Message	Signature (hexadecimal)
I owe you \$2000.	55A4E7F17F04CCFE2766E1EB32ADDBA890BBE92A6FBE2D785ED6E73CCB35E4CB
I owe you \$1500.	3BF7DFE62EDE9F5BFBA2A8C71539516DDD4F145087EF2C4EEA3B4FB87D85E607

What I observed

The two messages differ in 2 of their 16 bytes — 4 bits out of 128. The two signatures differ in **121 of their 256 bits**, about 47 percent, which is what you would expect from two unrelated random values rather than from two related ones. Modular exponentiation destroys any locality between input and output: there is no region of the signature that corresponds to the region of the message that changed.

The security consequence is that a signature cannot be patched. An attacker who intercepts a signed IOU and edits the amount cannot adjust the signature to match, because producing the new signature requires d . Verification recovers whatever integer the tampered signature encodes, and it will not be the altered message.

Caveat: signing the message directly, as this task requires, is not safe in practice. Raw RSA is multiplicative — the product of two signatures is a valid signature on the product of the messages — which allows existential forgery, and the message must also fit below the modulus. Real schemes sign a padded hash instead; see [RFC 8017 § 8 \(RSASSA-PSS and RSASSA-PKCS1-v1_5\)](#).

```
→ lab2 sudo gcc bn lab task4.c -lcrypto -o task4 && ./task4
ciphertext (hex): 55A4E7F17F04CCFE2766E1EB32ADDBA890BBE92A6FBE2D785ED6E73CCB35E4CB
plaintext (hex) 49206F776520796F752024323030302E
→ lab2 python3 -c 'print(bytes.fromhex("49206F776520796F752024323030302E").decode("ascii"))'
I owe you $2000.
→ lab2
→ lab2
→ lab2
→ lab2
→ lab2 python3 -c 'print("I owe you $1500.".encode("utf-8").hex())'
49206f776520796f752024313530302e
→ lab2 sudo gcc bn lab task4.c -lcrypto -o task4 && ./task4
ciphertext (hex): 3BF7DFE62EDE9F5BFBA2A8C71539516DDD4F145087EF2C4EEA3B4FB87D85E607
plaintext (hex) 49206F776520796F752024313530302E
→ lab2 python3 -c 'print(bytes.fromhex("49206F776520796F752024313530302E").decode("ascii"))'
I owe you $1500.
```

Figure 4. Both signing runs. The upper block signs `I owe you $2000.`; the lower block signs `I owe you $1500.` after the hex conversion. Each signature verifies back to its own message.

05 Verifying a signature

Bob receives $M = \text{"Launch a missile."}$ from Alice with signature S . Alice's key pair is different from the one used in Tasks 2 through 4; only her public key (e, n) is needed. Verification recovers $v = S^e \bmod n$ and compares it against the hexadecimal encoding of M :

```
BN_hex2bn(&n, "AE1CD4DC432798D933779FBD46C6E1247F0CF1233595113AA51B450F18116115");
BN_hex2bn(&e, "010001");
BN_mod_exp(v, S, e, n, ctx);    // v should equal M
```

The intact signature

```
v (hex): 4C61756E63682061206D697373696C652E
```

That decodes to `Launch a missile.`, which matches M exactly. Only the holder of Alice's private exponent could have produced a value that raises to this plaintext, so the signature is genuine.

The corrupted signature

Changing the final byte of S from `2F` to `3F` flips a single bit out of 256. Verification then returns:

```
v (hex): 91471927C80DF1E42C154FB4638CE8BC726D3D66C83A4EB6B7BE0203B41AC294
```

This is not valid ASCII — decoding it produces byte sequences such as `\xf1\xe4` and `\xcrm` with no printable structure — and it bears no resemblance to the expected message, so Bob rejects the signature.

What I observed

One flipped bit in the signature changed **123 of the 256 bits** of the recovered value, and lengthened it from 17 bytes to a full 32. Verification fails in the safest possible way: the result is not a near-miss that a lenient comparison might accept, but a value statistically indistinguishable from noise. That also rules out steering — an attacker cannot nudge a signature bit by bit toward a chosen message, because every change scatters the output completely.

```
* lab2 sudo gcc bn lab task5.c -lcrypto -o task5 && ./task5
v (hex): 4C61756E63682061206D697373696C652E
* lab2 python3 -c 'print(bytes.fromhex("4C61756E63682061206D697373696C652E").decode("ascii", errors="backslashreplace"))'
Launch a missile.
* lab2 sudo gcc bn lab task5.c -lcrypto -o task5 && ./task5
v (hex): 91471927C80DF1E42C154FB4638CE8BC726D3D66C83A4EB6B7BE0203B41AC294
* lab2 python3 -c 'print(bytes.fromhex("91471927C80DF1E42C154FB4638CE8BC726D3D66C83A4EB6B7BE0203B41AC294").decode("ascii", errors="backslashreplace"))'
\xf1\xe4,0\xb4c\x8c\xe8\xcrm=f\xc8:N\xb6\xb7\xbe\xb4\xc2\x94
```

Figure 5. Both verification runs. The upper block uses the intact signature and recovers the message; the lower block uses the one-bit corruption and recovers unprintable bytes.

Conclusion

Working through RSA on concrete numbers showed that a single primitive, modular exponentiation, does all four jobs. Which exponent is applied determines whether the operation is encryption, decryption, signing, or verification, and the security of all four rests on d being infeasible to derive from (e, n) without the factors of n — factors that Task 1 was handed outright. The signature tasks also showed why the textbook forms shown here are teaching tools rather than implementations: without padding, encryption is deterministic and direct signing is forgeable.

R Bibliography

Main background reading

Rivest, R. L., Shamir, A., & Adleman, L. (1978). [A Method for Obtaining Digital Signatures and Public-Key Cryptosystems](#). *Communications of the ACM*, 21(2), 120–126.

The original RSA paper sets out the key-generation procedure implemented in Task 1 and the observation that the encryption and decryption exponents are interchangeable, which is what makes the signature scheme in Tasks 4 and 5 possible. Its treatment of signatures assumes the direct form used in this lab; the padded constructions that replaced it came later.

Lab material

Du, W. (2018). [RSA Public-Key Encryption and Signature Lab](#). SEED Labs, Syracuse University.

Supporting technical references

The following documentation supports the descriptions of the `BIGNUM` API and of the padding schemes referenced in Tasks 2 and 4. Online documentation accessed September 20, 2026.

1. OpenSSL Project. [BN_new\(3\): Allocate and free BIGNUMs](#).
2. OpenSSL Project. [BN_CTX_new\(3\): Allocate and free BIGNUM temporary variables](#).
3. OpenSSL Project. [BN_add\(3\): Arithmetic operations on BIGNUMs](#), covering `BN_mul()`, `BN_sub_word()`, `BN_mod_exp()`, and `BN_gcd()`.
4. OpenSSL Project. [BN_mod_inverse\(3\): Compute inverse modulo n](#).
5. OpenSSL Project. [BN_bn2bin\(3\): Format conversions](#), covering `BN_hex2bn()`, `BN_bn2hex()`, and `BN_bn2dec()`.
6. Moriarty, K., Kaliski, B., Jonsson, J., & Rusch, A. (2016). [PKCS #1: RSA Cryptography Specifications Version 2.2](#) (RFC 8017). Internet Engineering Task Force.
7. die.net. [bn\(3\): Multiprecision integer arithmetic](#).